IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

<Juliane Oliveira Gonzaga dos Santos>
<8/23/2025>
This project used AI to assist with compiling and wording the results from the labs

Executive Summary

- Summary of methodologies
  - Exploratory Data Analysis (EDA) – to understand relationships between features.
  - Target Variable (class) – to see whether the landing was successful or not
  - scikit-learn's StandardScaler – to clean and standardize data.
  - Split data into training (80) and testing (20) split.
  - Trained using supervised learning models: Logistic Regression, Support Vector Machine (SVM), Decision Trees, and K-Nearest Neighbors (KNN)
  - Hyperparameter tuning with GridSearchCV – to identify optimal configurations (best results)

Executive Summary

Summary of results

The models showed some (but not considerable) variations between models.

Logistic Regression was the best model with a 85% of accuracy.

Decision Tree showed a bit of overfitting.

KNN showed good but a bit less accuracy than Logistic Regression.

Introduction

- **Project Background & Context**
- SpaceX has transformed space transportation. Before rockets would smash on groud, then part of it could be reused, decreasing costs of launching.

- The success of a Falcon 9 mission depends heavily on whether the first stage lands successfully.

- Landing success is influenced by multiple factors such as:
  - Payload mass
  - Orbit type
  - Launch site
  - Flight conditions

- Predicting landing success is crucial for:
  - Reducing costs
  - Supporting mission planning
  - Improving reliability

Introduction

Problems We Want to Answer

- Which features most strongly influence landing success?
- How to process and prepare launch data for machine learning?
- Which model performs best at predicting landings?
  - Logistic Regression
  - Support Vector Machines
  - Decision Trees
  - K-Nearest Neighbors
- What accuracy can we achieve on unseen test data?
- What do these results imply about the role of machine learning in aerospace?

Section 1

Methodology

Methodology

- Executive Summary
- Data Collection Methodology Data sourced from SpaceX APIs and public datasets (launches, payload, booster info).
- Data wrangling: handled missing values, standardized numeric features, encoded categorical variables.
- Exploratory Data Analysis (EDA): used SQL queries and visualizations to explore payload capacity, orbit trends, and booster reusability.
- Interactive Analytics:
- Folium → mapped launch sites and trajectories
- Plotly Dash → interactive dashboards for dynamic data exploration
- Predictive Analysis:
- Built classification models (Logistic Regression, SVM, Decision Tree, KNN)
- \Tuned hyperparameters using GridSearchCV with cross-validation
- Evaluated performance with test accuracy

Methodology

- EDA insights:
- Higher payload mass and orbit type strongly influence landing success
- Reusable boosters improved success rate across launches

- Model performance:
- Logistic Regression delivered best test accuracy (~85%)
- SVM and KNN performed closely, Decision Tree less stable (overfitting risk)

- Business Impact:
- Predictive models support forecasting booster recovery
- Insights contribute to cost reduction and launch reusability strategies

Data Collection

- Process Overview
- Source Identification: SpaceX API & public datasets
- Data Extraction: Collected data on launches, payloads, boosters, and landing outcomes
- Data Storage: Downloaded into CSV/JSON files for local processing
- Integration: Combined multiple sources (API + CSV) into a single structured dataset
- Preprocessing: Cleaned, standardized, and prepared data for analysis

Data Collection – SpaceX API

Data Collection with SpaceX REST API

Key Phrases (bullet points for clarity)

SpaceX REST API as the primary source of launch data

REST calls used to extract launch details (payload mass, booster, landing outcome)

JSON responses transformed into structured datasets (CSV/DF)

Data Wrangling performed to clean, filter, and standardize variables

Integrated dataset prepared for SQL, visualization, and ML modeling

[SpaceX REST API] → [REST Calls] → [JSON Response] → [Clean & Standardize Data] → [Unified Analytical Dataset]

https://github.com/julianegonzaga13/learningpython/blob/main/lab%201jupyter-labs-spacex-data-collection-api-v2.ipynb

Data Collection – Scraping

Target page: Wikipedia “List of Falcon 9 and Falcon Heavy launches”
•HTTP request: requests.get() → HTML response
•Parse HTML: BeautifulSoup → locate launch tables (table.wikitable)
•Header extraction: loop <th> → normalize column names
•Row parsing: iterate <tr>/<td> → clean text, drop footnotes/refs

•Value cleaning: helper funcs (e.g., get_mass, booster_version, landing_status)
•Structuring data: append to launch_dict → pandas DataFrame
•Quality checks: handle missing / multi-payload / multi-core rows
•Export: df.to_csv(...) for downstream wrangling/EDA/ML

Wikipedia URL (requests. get) → HTML Response (BeautifulSoup parse) → Find wikitable(s) (extract <th>, <tr>/<td>) → Clean & Normalize → (helpers: mass/orbit /landing) → launch_dict DataFrame → CSV / Ready for EDA & ML

https://github.com/julianegonzaga13/learningpython/blob/main/Lab%202%20jupyter-labs-webscraping.ipynb

Data Wrangling

Schema alignment: unified column names/types across API & scraped tables

Row filtering: single-core, single-payload launches; remove duplicates

Type casting: convert dates (date_utc → datetime), numeric fields (PayloadMass)

Missing values: impute PayloadMass with mean; retain LandingPad=None when no pad used

Feature extraction:

from rocket → BoosterVersion

from payload → PayloadMass, Orbit

from launchpad → site name, Lat, Long

from cores → Outcome, Landing type, Flights, GridFins, Reused, Legs, Block, ReusedCount, Serial, LandingPad

Target creation: Class = 1 for successful landing, 0 otherwise

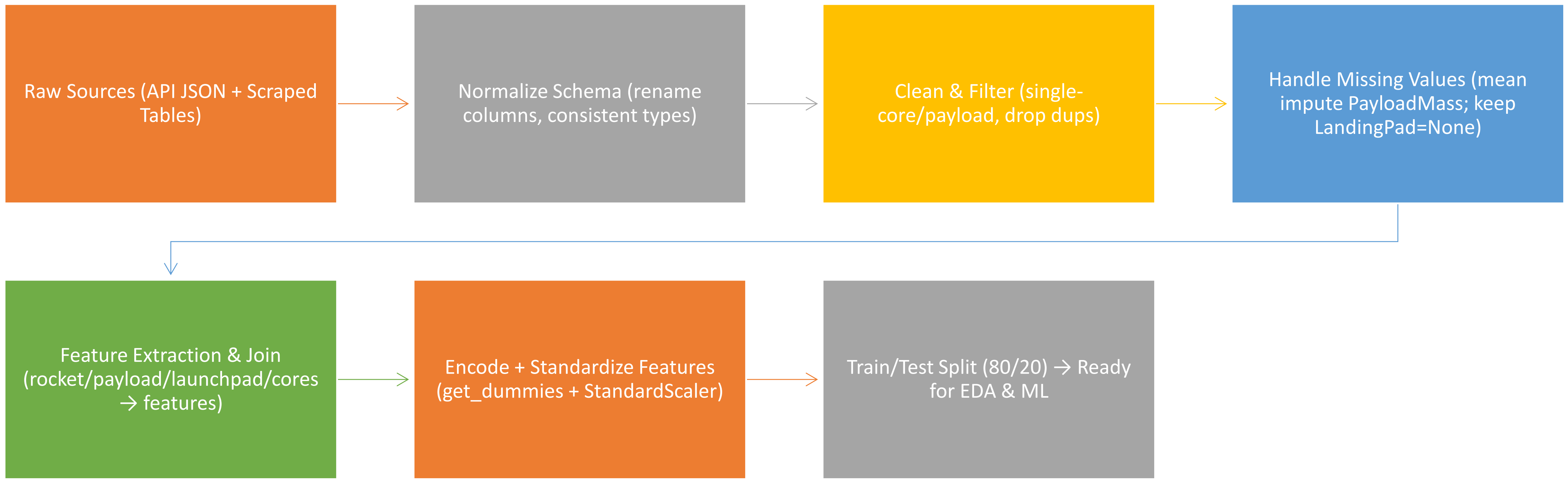
Encoding: one-hot for Orbit, LaunchSite, LandingPad, Serial

Standardization: scale numeric features with StandardScaler

Train/test split: 80/20 with stratification on Class

https://github.com/julianegonzaga13/learningpython/blob/main/Lab%203%20labs-jupyter-spacex-Data%20wrangling-v2.ipynb

Data Wrangling



EDA with Data Visualization

•Histograms (Payload Mass, Flight count, Block, Reused count, etc.)
•Why: To understand the distribution of key numeric features, detect skewness, and identify ranges where most launches occurred.
•Boxplots (Payload Mass vs. Orbit, Outcome vs. Flight/Block)
•Why: To explore relationships between categorical and numerical variables, detect outliers, and compare distributions across groups.

•Correlation Heatmap (Seaborn)
•Why: To check feature correlations, identify redundant variables, and guide feature selection for modeling.
•Scatter Plots (Payload Mass vs. Launch Success Probability)
•Why: To visualize trends and potential thresholds for successful landings depending on payload size.

https://github.com/julianegonzaga13/learningpython/blob/main/Lab%205%20jupyter-labs-eda-dataviz-v2.ipynb

EDA with Data Visualization

- Bar charts (Orbit type, Launch Site, Booster Version)
- Why: To measure categorical feature frequencies and compare them with landing success rates.
- Stacked Bar Charts (Orbit × Success, LandingPad × Outcome)
- Why: To evaluate categorical variables’ impact on the target (Class) and identify high-performing orbits/sites.
- Pair Plots (Seaborn)
- Why: To visualize multivariate relationships among selected features and their effect on success.

EDA with SQL

It was used: Select → Filter → Aggregate → Group → Subquery → Join → Conditional

Select & Filter

Queried launch records to retrieve specific columns such as *Launch Site*, *Orbit*, *Payload Mass*, and *Outcome*.

Filtered results for successful (class = 1) vs unsuccessful (class = 0) landings.

Aggregation Queries

Used COUNT() to determine the total number of launches per site.

Applied AVG() and MAX() to calculate average and maximum payload mass per orbit.

Applied MIN() to find the lightest payload launched.

Grouping & Ordering

Grouped by *Orbit* and *Launch Site* to compare performance metrics across categories.

Ordered results by *Payload Mass* or *Success Rate* to highlight top-performing orbits and sites.

https://github.com/julianegonzaga13/learningpython/blob/main/Lab%204%20jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA with SQL

- Subqueries
- Identified booster versions that carried the maximum payload mass by nesting queries with MAX(PayloadMass).
- Checked landing success rates by filtering on the maximum payload records.
- Join Operations
- Joined tables (where applicable) to combine *launch outcome data* with *site details* or *booster version data*.
- Conditional Queries
- Used CASE WHEN statements to label outcomes (e.g., “Success” vs. “Failure”).
- Queried launches above/below specific payload thresholds (e.g., WHERE PayloadMass > 4000).

Build an Interactive Map with Folium

Markers → Show launch site locations.

Circles → Highlight site areas.

Polylines → Connect launch sites to nearest coastlines.

Labels → Display calculated distances.

Clustering → Keep the map clean when markers are close together.

Purpose: To make the map both informative (locations + distances) and interactive (zoom, explore, clear visuals).

https://github.com/julianegonzaga13/learningpython/blob/main/LAB%206%20lab-jupyter-launch-site-location-v2.ipynb

Predictive Analysis (Classification)

- Define Target Variable → “Class” column (Landing Success: 1 or 0).
- Standardize Features → Scaled input data for fair model comparison.
- Split Data → Training set (80%) and Test set (20%).
- Build Models → Logistic Regression, Support Vector Machine, Decision Tree, K-Nearest Neighbors.
- Tune Hyperparameters → Used GridSearchCV with 10-fold Cross-Validation to optimize each model.
- Evaluate Performance → Measured accuracy on test data.
- Best Performing Model → Logistic Regression achieved ~85% test accuracy.

Predictive Analysis (Classification)

Data Prep
- Target = Class (0/1)
- Standardize features
- Train/Test split (80/20)

Model Candidates
- Logistic Regression
- SVM
- Decision Tree
- KNN

Hyperparameter Tuning
- GridSearchCV (cv=10)
- Accuracy (scoring)

Model Evaluation • Predict on test set
- Compare test accuracy

Best Model Selected
- Highest test accuracy
- Report best params
- Keep for deployment

Results

- Exploratory Data Analysis (EDA) Results
- Identified relationships between Payload Mass, Orbit type, and landing success (Class).
- Found that higher payloads reduced landing success probability.
- Launch sites differed in performance: some had consistently higher success rates.
- Correlation heatmaps highlighted key predictors (e.g., PayloadMass, Orbit, FlightNumber).

Results

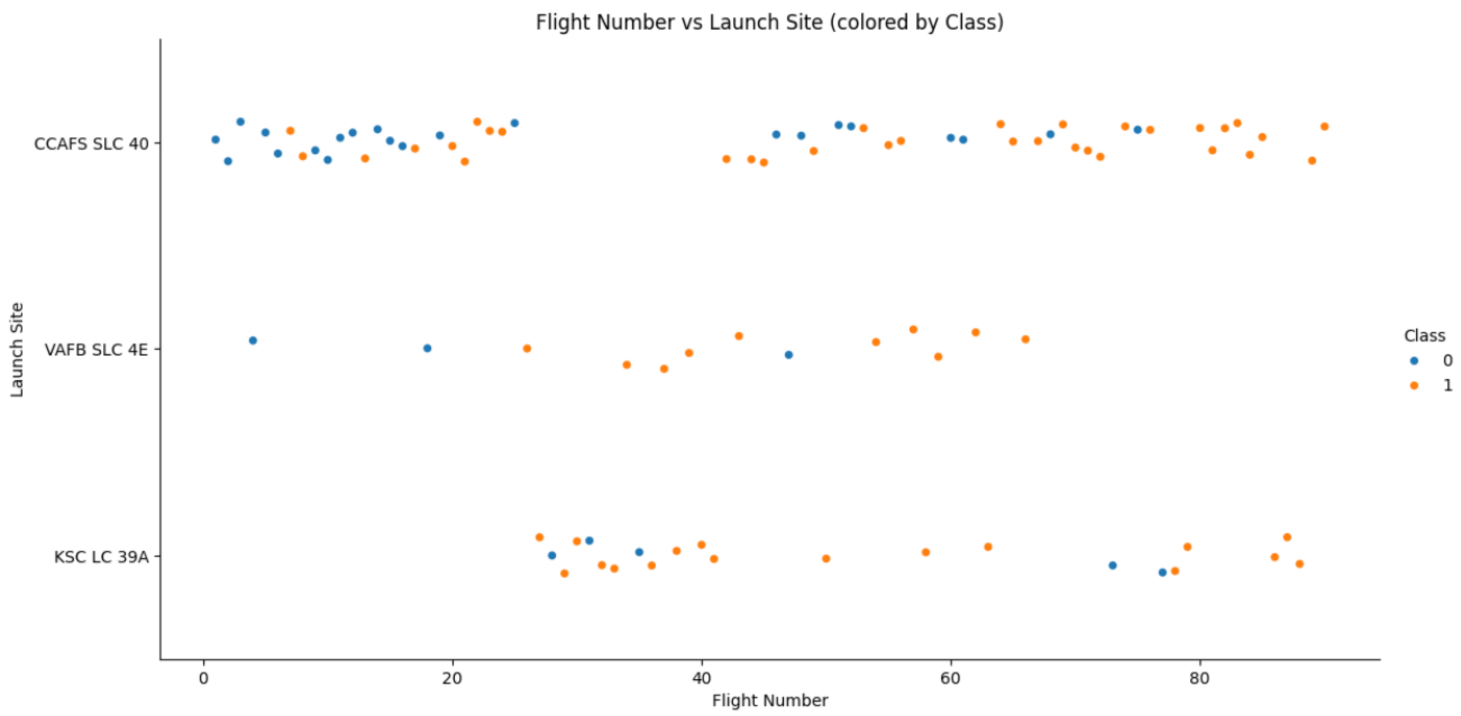
- Interactive Analytics Folium Map: Mapped launch sites with markers and distances to coastline/nearest cities. Added circles and polylines to visualize proximity and risk zones.

Results

- Predictive Analysis Results Models tested:
- Logistic Regression, SVM, Decision Tree, KNN.
- Hyperparameter tuning with GridSearchCV (cv=10) to optimize performance.
- Best validation accuracy: Logistic Regression (~85%).
- Final test accuracy: Logistic Regression also outperformed others → selected as best model. Strong indication that payload mass, orbit, and launch site are key features driving predictions.

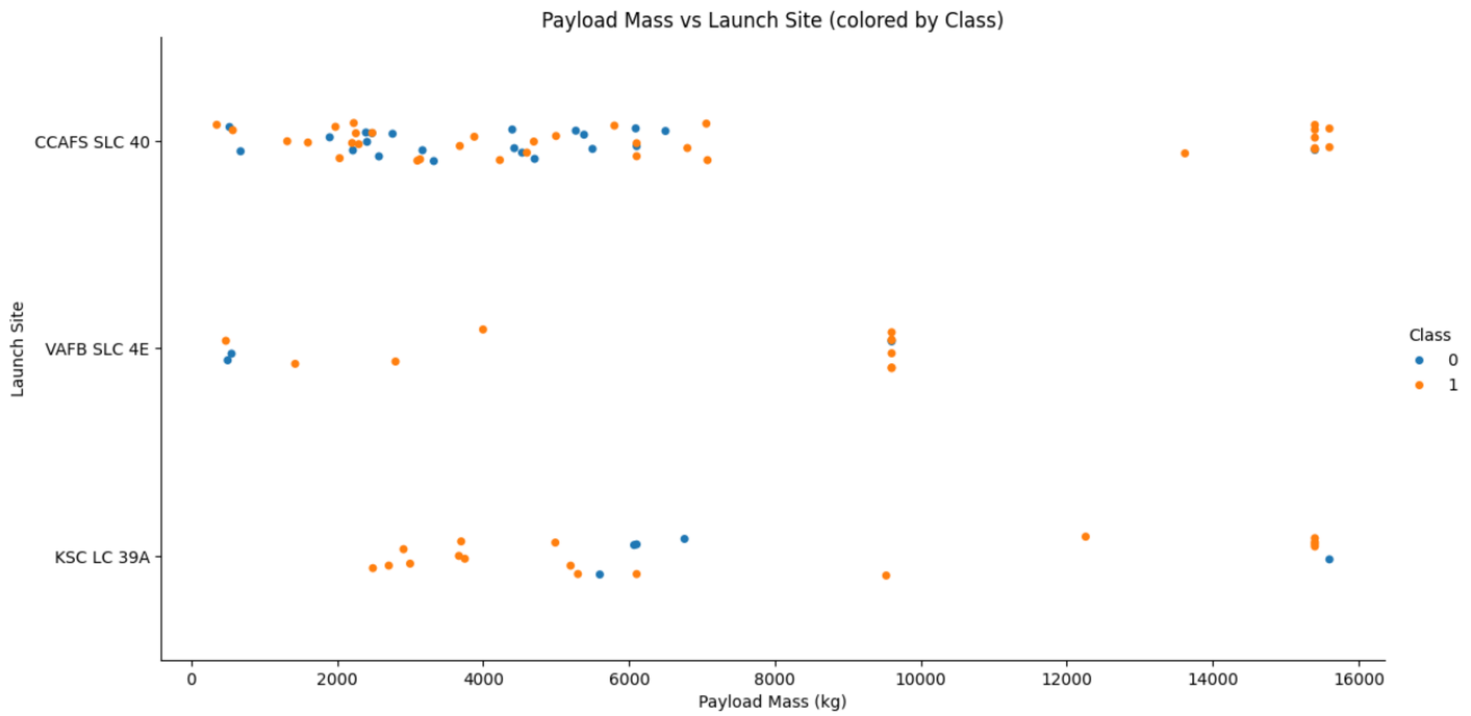
Section 2

Insights drawn from EDA



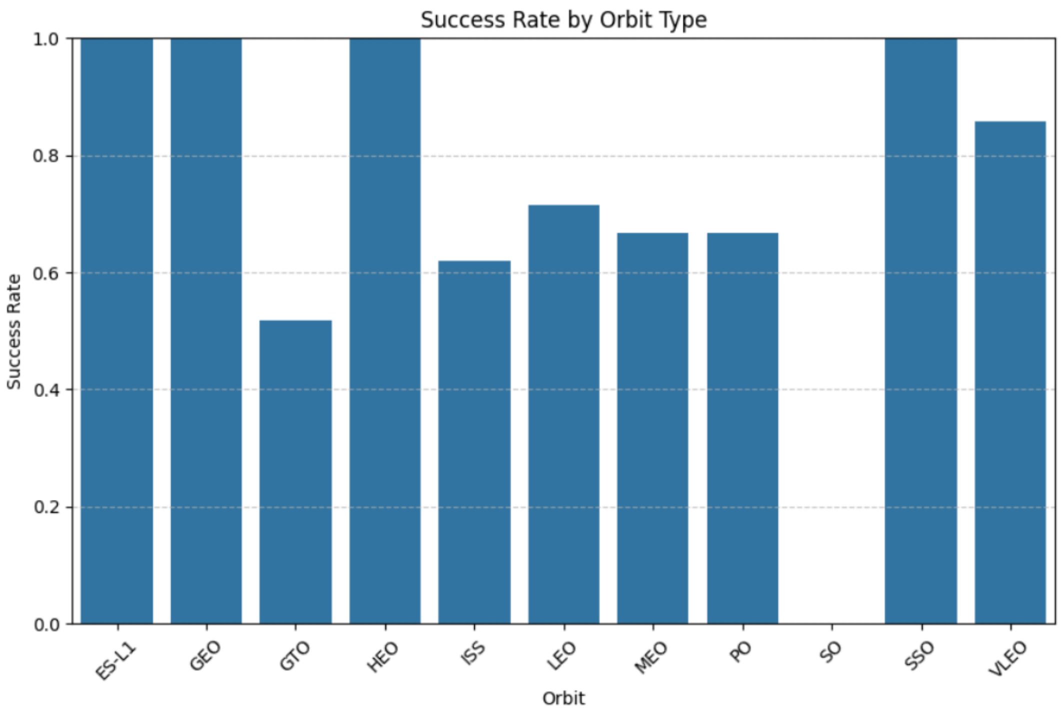
Flight Number vs. Launch Site

- The plot shows the relationship between flight number and launch site, with colors indicating landing success (orange = success, blue = failure). As the flight numbers increase, the proportion of successful landings also increases, especially at CCAFS SLC 40 and KSC LC 39A, while VAFB SLC 4E had fewer launches and mixed results.



Payload vs. Launch Site

- At CCAFS SLC 40, most launches succeed (orange) across a wide payload range, though failures (blue) occur at lighter to mid payloads. At VAFB SLC 4E, launches are fewer, but successes dominate. At KSC LC 39A, heavier payloads also show mostly successful landings, suggesting payload mass has less impact here.
- Success rates are high at all sites, but CCAFS SLC 40 shows more failures with mid-range payloads, while KSC LC 39A handles heavy payloads successfully.



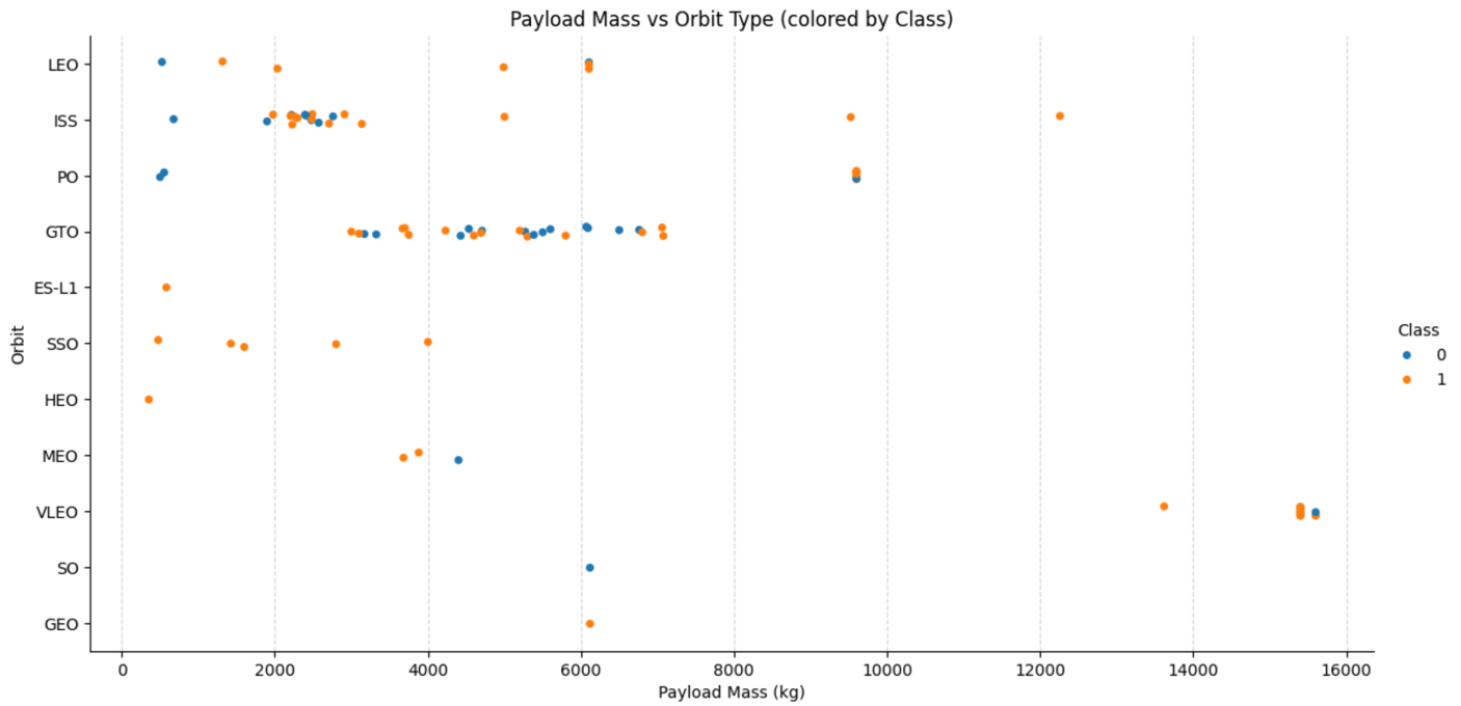
Success Rate vs. Orbit Type

- Some orbits like ES-L1, GEO, HEO, and SSO achieved a 100% success rate.
- Others like GTO, ISS, LEO, MEO, and PO show moderate success rates (50–70%).
- A few orbit types performed significantly better than others, indicating orbit choice impacts mission success.

- Certain orbits (ES-L1, GEO, SSO) had perfect success, while GTO and some others showed lower reliability.

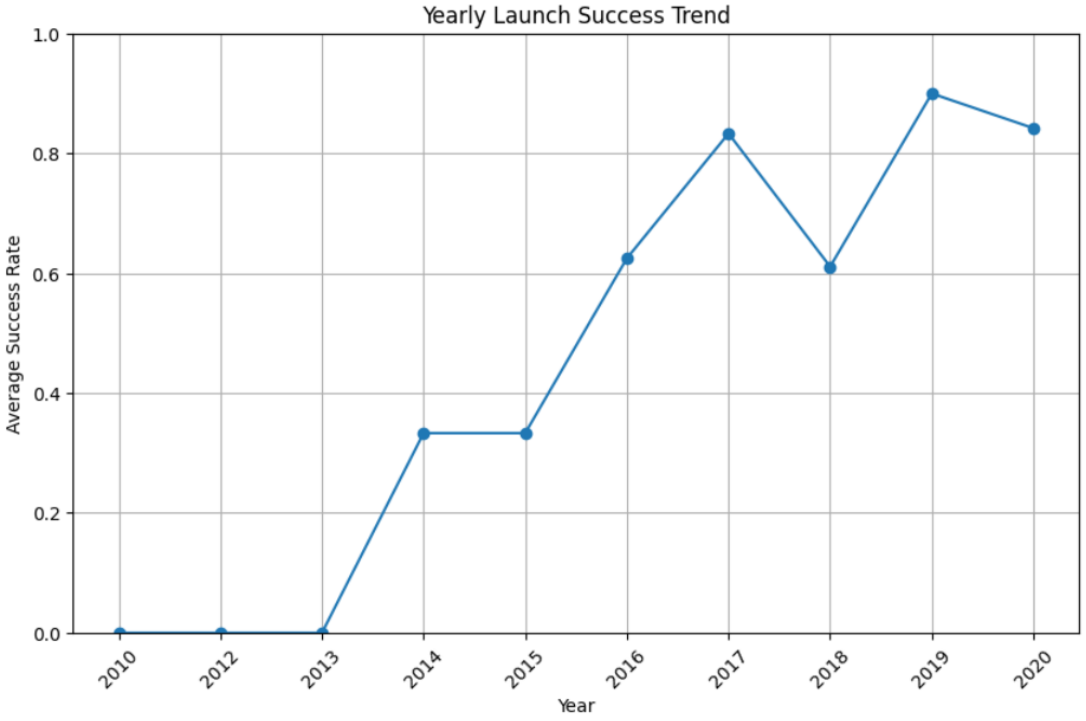
Flight Number vs. Orbit Type

- **Trend**: Successes (orange) increase with higher flight numbers.
- **Orbit Insight**: Early flights had more failures (blue), but later missions across multiple orbit types achieved higher success.
- **Finding**: Experience (flight count) improves reliability across orbits.



Payload vs. Orbit Type

- Most payloads cluster between 2,000–6,000 kg.
- Heavier payloads (above 10,000 kg) are less frequent but still mostly successful.
- Successes (orange) are spread across orbit types, while failures (blue) are fewer and scattered.
- Certain orbits (ISS, LEO, GTO) handle a wide range of payloads, while others (VLEO, HEO, ES-L1) have smaller samples.
- Higher payloads generally succeed, with some failures concentrated in mid-range orbits.



Launch Success Yearly Trend

- This chart shows the trend of SpaceX's launch success rate from 2010 to 2020:
- Early years (2010–2013): Very low success rate (close to zero), reflecting the challenges of initial launches.
- 2014–2016: Noticeable improvement, reaching over 60% success by 2016.
- 2017–2019: Strong upward trend, peaking at around 90% success in 2019.
- 2020: Slight dip but overall consistently high success rate (>80%).
- SpaceX demonstrated steady progress and reliability improvements over the decade, transitioning from frequent failures to near-consistent success.

Task 1

Display the names of the unique launch sites in the space mission

```
%sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

All Launch Site Names

- This SQL query retrieves the unique launch site names from the SPACEXTABLE.
- It lists all distinct launch sites used in the missions.

Display 5 records where launch sites begin with the string 'CCA'

```
%sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Launch Site Names Begin with 'CCA'

- This SQL query selects the first 5 records where the launch site name starts with "CCA".
- The LIKE 'CCA%' condition filters launch sites beginning with "CCA".
- The query outputs mission details such as date, time, booster version, payload, mass, orbit, customer, mission outcome, and landing outcome.
- From the results, we see that all 5 missions were launched at "CCAFS LC-40" with varying payloads and outcomes.
- This query filters and displays SpaceX missions launched from Cape Canaveral (CCAFS), showing early launches with mixed payload masses and landing outcomes.

Display the total payload mass carried by boosters launched by NASA (CRS)

```
%sql SELECT SUM("Payload_Mass__kg_") AS Total_Payload_Mass FROM SPACEXTABLE WHERE "Customer" LIKE '%NASA (CRS)%';
```

```
 * sqlite:///my_data1.db
Done.
```

Total_Payload_Mass
48213

Total Payload Mass

- This query calculates the total payload mass carried by boosters launched for NASA (CRS) missions.
- The SQL command uses SUM("Payload_Mass__kg_") to add up all payload weights.
- The WHERE condition ensures only rows where the customer is NASA (CRS) are included.
- The result shows 48,213 kg, meaning NASA CRS launches carried a combined payload mass of 48.2 tons.

Display average payload mass carried by booster version F9 v1.1

```
%sql SELECT AVG("Payload_Mass__kg_") AS Average_Payload_Mass FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';
```

```
 * sqlite:///my_data1.db
Done.
```

Average_Payload_Mass
2928.4

Average Payload Mass by F9 v1.1

- This SQL query calculates the average payload mass carried specifically by the booster version F9 v1.1.
- Explanation:
- It uses AVG() to compute the mean payload mass.
- The condition filters only records where the Booster_Version is 'F9 v1.1'.
- The result shows that the average payload mass for F9 v1.1 boosters is 2928.4 kg.
- This query helps us understand the typical payload capacity of the F9 v1.1 booster across its launches.

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

```
%sql SELECT MIN("Date") AS First_Successful_Ground_Landing FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';
```

```
 * sqlite:///my_data1.db
Done.
```

First_Successful_Ground_Landing
2015-12-22

First Successful Ground Landing Date

This query finds the earliest date when a booster had a successful ground pad landing.

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (drone ship)' AND "Payload_Mass__kg_" > 4000 AND "Payload_Mass__kg_"
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Successful Drone Ship Landing with Payload between 4000 and 6000

- This query filters boosters that successfully landed on a drone ship and carried a payload mass between 4000–6000 kg.
- **Result:** The boosters meeting these conditions are **F9 FT B1022, B1026, B1021.2, and B1031.2**.
- *It shows the booster versions with drone ship landing success and medium–heavy payloads.*

List the total number of successful and failure mission outcomes

```
%sql SELECT "Landing_Outcome", COUNT(*) AS Total FROM SPACEXTABLE GROUP BY "Landing_Outcome";
```

```
 * sqlite:///my_data1.db
Done.
```

Landing_Outcome	Total
Controlled (ocean)	5
Failure	3
Failure (drone ship)	5
Failure (parachute)	2
No attempt	21
No attempt	1
Precluded (drone ship)	1
Success	38
Success (drone ship)	14
Success (ground pad)	9
Uncontrolled (ocean)	2

Total Number of Successful and Failure Mission Outcomes

- Key Findings:
- Successful landings:
  - Ground pad: 9
  - Drone ship: 14
  - General success: 38
- Failures:
  - Drone ship: 5
  - Parachute: 2
  - Other failures: 3
- Other outcomes:
  - Controlled ocean: 5
  - Uncontrolled ocean: 2
  - No attempt: 22
  - Precluded: 1
- SpaceX achieved more successes (61 total) than failures, with a mix of attempts and non-attempts.

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
%sql SELECT "Booster_Version", "Payload_Mass__kg_" FROM SPACEXTABLE WHERE "Payload_Mass__kg_" = (SELECT MAX("Payload_Mass__kg_") FROM SPACEXTABLE);
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version	PAYLOAD_MASS__KG_
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

Boosters Carried Maximum Payload

- This query finds all boosters that carried the maximum payload mass by comparing each record’s payload with the maximum value in the table (using a subquery).
- Result:
Multiple boosters (all F9 B5 series) carried the maximum payload of 15,600 kg.
- *Shows every booster that achieved the heaviest recorded payload.*

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
%sql SELECT substr("Date", 6, 2) AS Month, "Landing_Outcome", "Booster_Version", "Launch_Site" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Failure (dro
```

```
 * sqlite:///my_data1.db
Done.
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

2015 Launch Records

- This query extracts failures on drone ships in 2015, showing the month, landing outcome, booster version, and launch site.
- Output:
- January (01) 2015 → Failure (drone ship), Booster F9 v1.1 B1012, CCAFS LC-40
- April (04) 2015 → Failure (drone ship), Booster F9 v1.1 B1015, CCAFS LC-40
- It lists 2015 drone ship landing failures with their month, booster, and site.

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%sql SELECT "Landing_Outcome", COUNT(*) AS Outcome_Count FROM SPACEXTABLE WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome"
```

```
 * sqlite:///my_data1.db
Done.
```

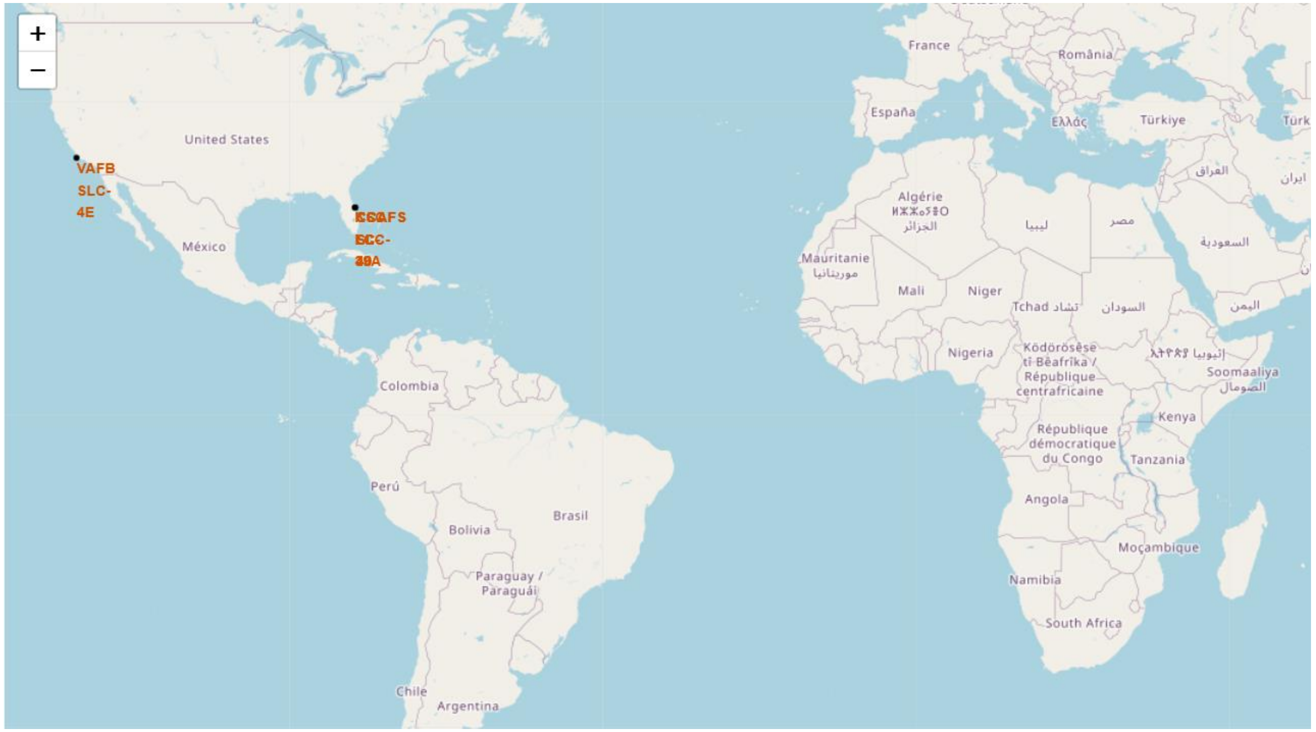
Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- This query ranks different landing outcomes between 2010-06-04 and 2017-03-20.
- Explanation
- It groups the results by Landing_Outcome.
- Counts the number of times each outcome happened.
- Orders them by count in descending order.
- Key Insights
- Most frequent outcome: "No attempt" (10 times).
- Both Success (drone ship) and Failure (drone ship) occurred 5 times each.
- Other outcomes like ground pad success, ocean controlled/uncontrolled, parachute failures, and precluded had lower counts.
- The majority were "No attempt," and drone ship landings had equal numbers of success and failure within this timeframe.

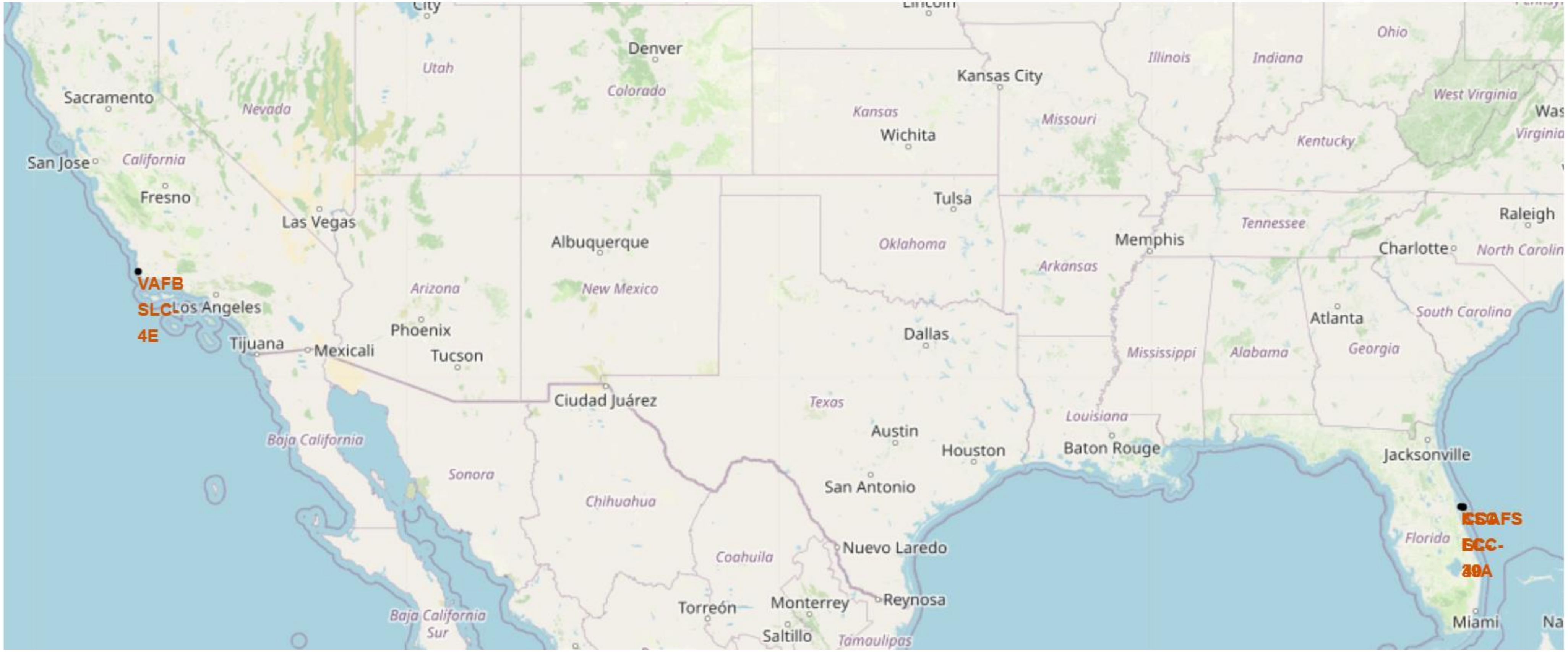
Section 3

Launch Sites Proximities Analysis



Launch Sites

- Launch Site Markers → Added markers for all major SpaceX launch sites.
- Interactive Popups → Displayed site names and details for quick comparison. A zoom would be necessary.



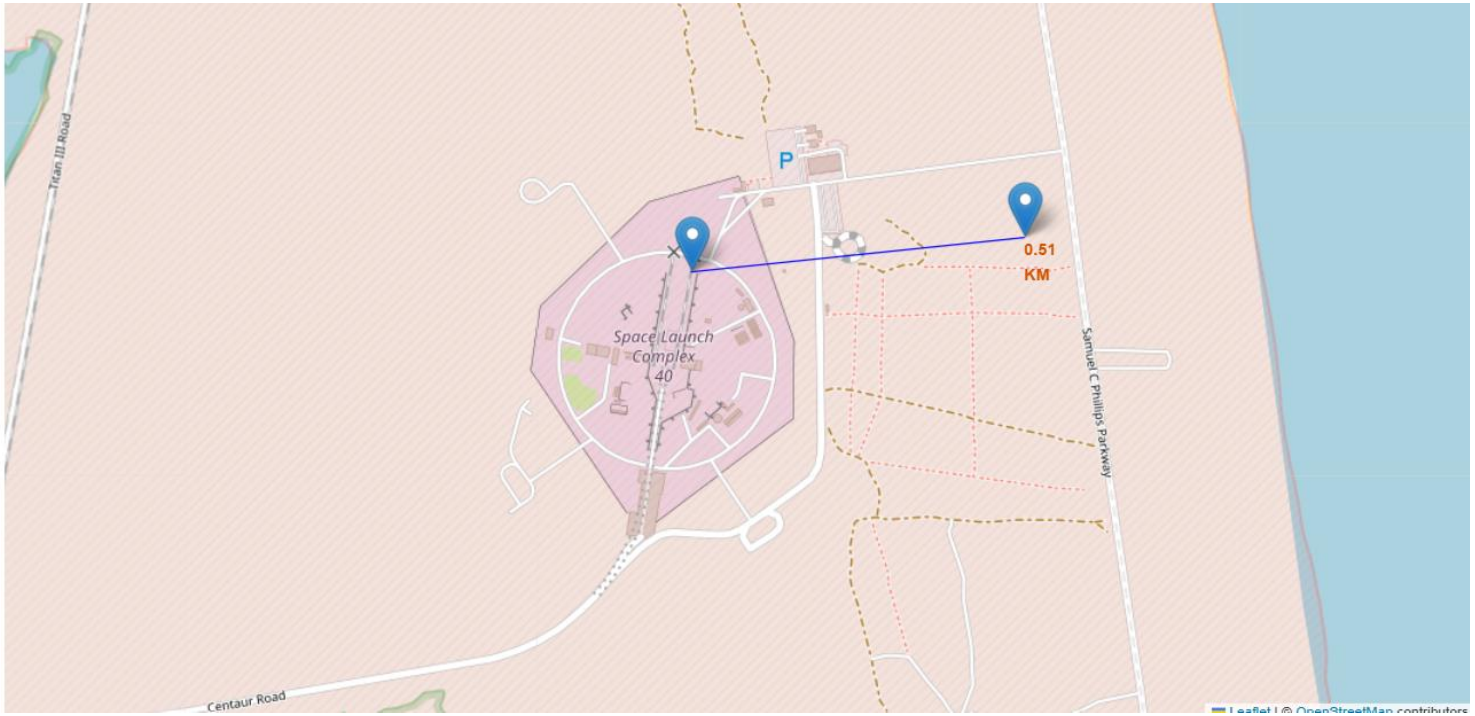
Launch Sites MAP

Launch Sites MAP

Folium Map

- Folium Map – Key Elements & Findings
- Coastline Proximity → Used circles/lines to show distance to coastlines for safety analysis.
- Findings:
- Launch sites are strategically coastal, minimizing risk to populations.
- Mapping confirms safety and recovery considerations.
- Interactive map enables easy exploration of site locations and characteristics.

Distance from Launch Site



Conclusion

- I collected via APIs the data from SpaceX, did some web scraping, and processed with wrangling.
- I explored the dataset by running EDA with SQL queries and visualizations to spot patterns. I also mapped using Folium to visualize launch sites and distances.
- I could notice that Launch success depends strongly on payload mass, orbit type, and site.
- The Modeling Results showed that the Logistic Regression performed best (~85% accuracy).
- I could conclude that Data science methods can reliably predict rocket landings and generate real business insights.

Thank you!